AGENDA 2

UTILIZANDO MICROSOFT MAUI PARA DESENVOLVIMENTO MOBILE

GEEaD - Grupo de Estudos de Educação a Distância

Centro de Educação Tecnológica Paula Souza

GOVERNO DO ESTADO DE SÃO PAULO

EIXO TECNOLÓGICO DE INFORMAÇÃO E COMUNICAÇÃO

CURSO TÉCNICO EM DESENVOLVIMENTO DE SISTEMAS

PROGRAMAÇÃO MOBILE II

Expediente

Autores:

KELLY CRISTIANE DE OLIVEIRA DAL POZZO

TIAGO ANTONIO DA SILVA

Revisão Técnica:

GUILHERME HENRIQUE GIROLLI

São Paulo – SP, 2024

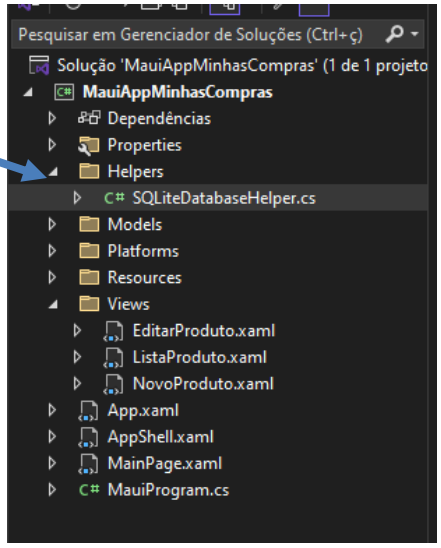
Na agenda anterior, implementamos a base do aplicativo, garantindo a persistência de dados utilizando SQLite. Agora, vamos avançar para a criação da tela de listagem de produtos, permitindo a exibição dos dados armazenados no banco. Além disso, vamos aprofundar a implementação do CRUD (Create, Read, Update, Delete), fundamental para manipular as informações.

O CRUD permite que possamos inserir, visualizar, atualizar e excluir produtos, tornando o aplicativo funcional e interativo. Nesta agenda, vamos focar nas operações de leitura e inserção de produtos, garantindo que o usuário possa visualizar os itens cadastrados de forma dinâmica.

Classe SQLiteDatabaseHelper

A comunicação entre a classe SQLiteDatabaseHelper e o banco de dados SQLite é essencial para a persistência de dados no aplicativo. Essa classe é responsável por garantir que os produtos sejam armazenados corretamente e possam ser recuperados posteriormente.

No diretório Helpers adicione a classe SQLiteDatabaseHelper.



A versão final da classe deve ficar da seguinte forma:

```
using MauiAppMinhasCompras.Models;
using SQLite;

namespace MauiAppMinhasCompras.Helpers
{
    public class SQLiteDatabaseHelper
    {
        readonly SQLiteAsyncConnection _conn;

        public SQLiteDatabaseHelper(string path)
        {
            _conn = new SQLiteAsyncConnection(path);
```

```
            _conn.CreateTableAsync<Produto>().Wait();
        }

        public Task<int> Insert(Produto p)
        {
            return _conn.InsertAsync(p);
        }

        public Task<List<Produto>> Update(Produto p)
        {
            string sql = "UPDATE Produto SET Descricao=?, Quantidade=?, Preco=? WHERE Id=?";

            return _conn.QueryAsync<Produto>(
                sql, p.Descricao, p.Quantidade, p.Preco, p.Id
            );
        }

        public Task<int> Delete(int id)
        {
            return _conn.Table<Produto>().DeleteAsync(i => i.Id == id);
        }

        public Task<List<Produto>> GetAll()
        {
            return _conn.Table<Produto>().ToListAsync();
        }

        public Task<List<Produto>> Search(string q)
        {
            string sql = "SELECT * Produto WHERE descricao LIKE '%" + q + "%'";

            return _conn.QueryAsync<Produto>(sql);
        }
    }
}
```

Agora vejamos em detalhes o funcionamento de cada um dos métodos:

```
readonly SQLiteAsyncConnection _conn;
```

O modificador readonly significa que o campo só pode ser atribuído no momento da declaração ou dentro do construtor da classe. Isso ajuda a garantir que a conexão do banco de dados não seja substituída acidentalmente. A classe SQLiteAsyncConnection é usada para gerenciar operações com o banco de dados de maneira assíncrona. Assim temos o atributo _conn utilizado para gerenciar as operações no banco.

```
        public SQLiteDatabaseHelper(string path)
        {
            _conn = new SQLiteAsyncConnection(path);
            _conn.CreateTableAsync<Produto>().Wait();
        }
```

O construtor SQLiteDatabaseHelper inicializa a conexão com o banco de dados e garante que a tabela necessária (Produto) seja criada assim que a instância do helper for criada.

O método **_conn.CreateTableAsync<Produto>().Wait();** cria, de forma assíncrona, uma tabela chamada Produto, com base na classe modelo Produto. Se a tabela já existir, ele não faz nada (SQLite trata isso automaticamente). O comando **Wait()** é chamado para bloquear a execução do construtor até que a tabela seja criada, garantindo que o banco esteja configurado antes de outras operações.

Método Insert

```
public Task<int> Insert(Produto p)
        {
            return _conn.InsertAsync(p);
        }
```

Gerencia operações com o banco de dados SQLite de forma assíncrona, este método insere um registro do tipo Produto no banco de dados SQLite.

```
Task<List>
```

A assinatura Task retorna um valor inteiro que representa o número de linhas afetadas pela operação de inserção (normalmente será 1 se a inserção for bem-sucedida).

O metodo Insert será executado, seguindo os passos:

1- O método Insert é chamado com uma instância de Produto como argumento.

2- var produto = new Produto { Nome = "Produto A", Preco = 10.5 };

3- await databaseHelper.Insert(produto);

4- O método InsertAsync é chamado internamente, passando o objeto produto para o banco de dados.

5- O SQLite cria uma nova linha na tabela correspondente à classe Produto e adiciona elementos a essa linha com os valores das propriedades do objeto produto.

Método Update

O método Update tenta atualizar um registro existente na tabela Produto no banco de dados, com base no ID fornecido no objeto Produto (p). A consulta SQL atualiza as colunas Descricao, Quantidade e Preco do registro que corresponde ao Id especificado.

```
        public Task<List<Produto>>
```

O método retorna uma tarefa assíncrona (Task) que resolve em uma lista de objetos Produto (List<Produto>).

No entanto, para uma operação de UPDATE, o retorno esperado geralmente seria um inteiro (número de linhas afetadas), e não uma lista de produtos. Isso sugere um erro de design ou entendimento no método.

```
UPDATE Produto SET Descricao=?, Quantidade=?, Preco=? WHERE Id=?"
```

UPDATE Produto: Atualiza a tabela chamada Produto.

SET Descricao=?, Quantidade=?, Preco=?: Define os novos valores para as colunas Descricao, Quantidade e Preco.

WHERE Id=?: Restringe a atualização ao registro cujo Id corresponde ao valor fornecido.

?: São placeholders para valores que serão substituídos pelos parâmetros fornecidos.

Método Delete

O método Delete remove um registro da tabela Produto com base no Id fornecido como argumento. Ele utiliza a funcionalidade assíncrona do SQLite para realizar a exclusão sem bloquear a thread principal.

```
        public Task<int> Delete(int id)
```

public: O método é público e pode ser chamado de fora da classe onde está implementado.

Task<int>: Retorna uma tarefa assíncrona que resolve em um inteiro (int).

O valor retornado indica o número de linhas afetadas pela exclusão (geralmente, 1 se a exclusão foi bem-sucedida ou 0 se nenhum registro foi encontrado).

```
return _conn.Table<Produto>().DeleteAsync(i => i.Id == id);
```

Apenas registros onde o campo Id do produto (i.Id) corresponde ao valor fornecido no argumento (id) serão excluídos.

O metodo Delete será executado, seguindo os passos:

1- O método é chamado com o ID do registro que deve ser excluído.

2- `await databaseHelper.Delete(3);`

3- A biblioteca SQLite-net-pcl busca o registro na tabela Produto cujo Id seja igual a 3.

4- Se um registro correspondente for encontrado, ele será excluído do banco de dados.

5- O método retorna o número de linhas afetadas pela operação (normalmente 1 para uma exclusão bem-sucedida ou 0 se nenhum registro for encontrado).

Método GetAll

O método GetAll é utilizado para consulta a tabela Produto no banco de dados e retorna todos os registros na forma de uma lista de objetos Produto. A operação é assíncrona, permitindo que o método seja executado sem bloquear a thread principal.

```
public Task<List<Produto>> GetAll()
```

Indica que o método é assíncrono e retorna uma tarefa (Task) que resolve em uma lista (List<Produto>), isso permite que o chamador do método use await para aguardar o resultado.

```
return _conn.Table<Produto>().ToListAsync();
```

Representa uma consulta que acessa a tabela Produto no banco de dados. É equivalente a "SELECT * FROM Produto" em SQL.

O comando ToListAsync() executa a consulta e converte o resultado em uma lista assíncrona (List<Produto>).

A operação é realizada em segundo plano e retorna todos os registros da tabela.

O metodo GetAll será executado, seguindo os passos:

1- Quando o método é chamado, ele utiliza a conexão _conn (uma instância de SQLiteAsyncConnection) para acessar a tabela Produto.

2- Ele cria uma consulta que seleciona todos os registros dessa tabela (_conn.Table<Produto>()).

3- O método ToListAsync() executa a consulta e converte os resultados em uma lista de objetos Produto.

4- O resultado é retornado como uma tarefa (Task<List<Produto>>), que pode ser aguardada com await.

Método Search

O método Search implementa uma funcionalidade de busca na tabela Produto do banco de dados SQLite, permitindo filtrar registros de acordo com uma string fornecida. A busca é realizada usando uma consulta SQL que utiliza o operador LIKE para encontrar produtos cujas descrições contenham a string q, sendo:

- O método Search realiza uma busca na tabela Produto, procurando por produtos cuja descrição (descricao) contenha a string fornecida como parâmetro (q).
- A operação é realizada de forma assíncrona, ou seja, ela será executada em segundo plano, não bloqueando a thread principal.

```
public Task<List<Produto>> Search(string q)
```

O método publico retorna um Task que, ao ser completado, resulta em uma lista de objetos Produto (List<Produto>). O uso de Task indica que o método é assíncrono e o retorno será resolvido em algum momento no futuro. O parâmetro q é a string de busca fornecida pelo usuário, que será usada para filtrar as descrições dos produtos.

```
string sql = "SELECT * Produto WHERE descricao LIKE '%" + q + "%'";
```

SELECT * :Seleciona todos os campos da tabela Produto.

FROM Produto: Indica que a consulta será realizada na tabela Produto.

WHERE descricao LIKE '%" + q + "%': Aplica um filtro à coluna descricao, procurando por registros que contenham a string q em qualquer parte da descrição.

O operador LIKE permite buscas parciais, sendo:

%: Os caracteres % são usados como coringas, indicando que qualquer coisa pode aparecer antes ou depois da string q.

Exemplo: Se q = "cadeira", a consulta seria:

SELECT * FROM Produto WHERE descricao LIKE '%cadeira%'

Isso retornaria todos os produtos cujas descrições contêm a palavra "cadeira", como "Cadeira de escritório", "Cadeira de praia", etc.

```
return _conn.QueryAsync<Produto>(sql);
```

O método QueryAsync da biblioteca SQLite-net-pcl é utilizado para executar consultas SELECT e retorna os dados em uma lista de objetos do tipo Produto.

O metodo Search será executado, seguindo os passos:

1- O método Search é chamado com uma string de busca:

   a. `string busca = "cadeira";`

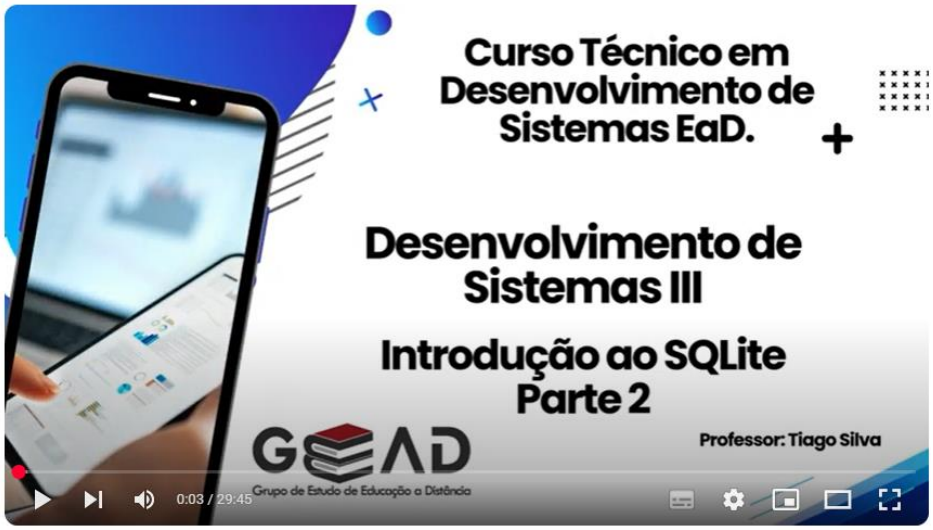
2- `List<Produto> produtos = await databaseHelper.Search(busca);`

3- A consulta SQL é construída dinamicamente, inserindo a string q na consulta com o operador LIKE.

4- A consulta é executada de forma assíncrona, e os resultados (todos os produtos cujas descrições contêm a string q) são retornados em uma lista de objetos Produto.

5- O método retorna um Task<List<Produto>>, que, ao ser aguardado com await, fornecerá os resultados da pesquisa.

Assista o vídeo apresentando o passo a passo para criação do aplicativo:



Disponível em: https://www.youtube.com/watch?v=OZRofDwVB7c

CÓDIGO-FONTE DA AGENDA

https://github.com/tiagotas/MauiAppMinhasCompras/tree/c1ff29274690d6a15d85a67d4580c493a0aa39c8